AI-Driven Self-Healing Test Automation: A DOM-Similarity and Machine-Learning Framework for Resilient Web UI Regression Testing

Vijay P. Javvadi
Independent Researcher
Email: research@vijayjavvadiresearch.ai
LinkedIn: linkedin.com/in/vijayjavvadi
Website: vijayjavvadiresearch.ai

April 21, 2026

Abstract

Modern software systems evolve rapidly, and automated user-interface (UI) test scripts frequently fail due to minor front-end changes, dynamic element identifiers, or evolving application structures. Traditional automation frameworks require continuous manual maintenance to update broken locators and test scripts, resulting in increased maintenance cost, slower release cycles, and reduced confidence in regression coverage. This paper proposes an AI-driven self-healing test automation framework capable of automatically detecting, diagnosing, and repairing failing test cases caused by UI changes. The framework integrates a structured locator metadata store, Document Object Model (DOM) similarity analysis, heuristic candidate ranking, and a machine-learning ensemble to dynamically identify alternative elements when original locators become invalid. At runtime, the framework monitors historical execution data and page-structure patterns to generate predictive locator models that enable automated recovery during test execution. When a test step fails because of an element change, the framework evaluates multiple candidate elements using structural similarity metrics, attribute matching, and machine-learning-assisted ranking to determine the most probable replacement element.

To evaluate the approach, we constructed a reproducible, mutation-based experimental pipeline that derives five progressively mutated versions of a representative web application, applying seven mutation categories—identifier change, attribute removal, DOM wrapping, element reordering, class change, text change, and placeholder change—across fifty-two total mutation events. Nine Selenium-based test suites were executed across all versions under three ranking modes (heuristic, ML, and hybrid), producing a dataset of 1,600 experiment rows per mode, 2,400 broken locator events, and up to 1,750 healed events. The empirical evaluation yields a heuristic healing success rate (HSR) of 64.58% with a Wilson 95% confidence interval of [62.65%, 66.47%].

A gradient-boosting ML ranker achieves 65.31% HSR [63.40%, 67.17%], and a hybrid ensemble combining heuristic and ML scores achieves **68.63% HSR** [66.80%, 70.40%], a statistically significant improvement of +4.05 percentage points over the heuristic baseline. Per-version analysis reveals that the hybrid ranker delivers its largest lift on the hardest mutation set (version_4): 58.3% → 73.3%, a +15 percentage-point gain. A 13-feature gradient-boosting classifier trained on 7,750 labeled examples achieves AUC = 0.9993 and F1 = 0.9768 on held-out data, confirming that the ML signal is both learnable and complementary to heuristic scoring. A multi-model comparison further evaluates XGBoost against Gradient Boosting: XGBoost achieves higher classification metrics (AUC = 0.9994, Precision = 1.0000) and 3.5× faster training, but its conservative recall (0.929 vs. 0.965) yields a lower standalone ML HSR of 61.70%. The hybrid ensemble equalises both algorithms at 68.63% HSR, demonstrating that the α-blending mechanism compensates for per-algorithm biases. A denominator decomposition shows that 100% of broken events produced a ranked candidate, meaning the 35.42% unresolved share in the heuristic mode reflects a recovery-executor bottleneck rather than a ranking bottleneck. The results demonstrate that AI-driven locator recovery substantially reduces manual automation maintenance while significantly improving test execution continuity across evolving application versions.

Keywords: Self-healing automation, UI testing, Selenium, DOM similarity, machine learning, gradient boosting, hybrid ranking, web test resilience, locator recovery, mutation testing, regression testing, intelligent quality engineering

1 Introduction

Automated web UI testing is a foundational quality-assurance activity in modern software engineering. Functional regression suites are routinely executed in continuous integration and continuous delivery (CI/CD) build pipelines to validate navigation flows, input handling, shopping-cart behaviour, authentication scenarios, and checkout interactions. In principle, UI automation reduces manual test effort, improves repeatability, and increases release confidence. In practice, however, automated UI tests remain strikingly sensitive to changes in front-end implementation details. Industry surveys and developer studies consistently identify locator maintenance as one of the most expensive and time-consuming activities in large regression environments, and a non-trivial share of "broken" test runs in production CI pipelines do not reflect any defect in application behaviour. Instead, they indicate that an element identifier, attribute, or DOM path has shifted.

The central source of this sensitivity is *locator fragility*. Selenium-based tests identify interface elements through identifiers such as XPath expressions, CSS selectors, element IDs, placeholder values, and text attributes. These locators encode strong assumptions about the structure of the current DOM. When a front-end developer changes element attributes, inserts wrapper containers, replaces CSS classes, or reorganises page hierarchy, tests may fail even when the business behaviour remains correct. A significant fraction of UI test failures therefore represent automation maintenance issues rather than true application defects.

This problem becomes more severe in rapidly changing web systems. Teams practising continuous delivery frequently modify front-end frameworks, component libraries, and responsive layouts. A locator valid in one release may fail in the next not because the user

journey is broken, but because the DOM no longer matches the assumptions embedded in the script. The cost of repeatedly repairing locators across large test suites is substantial, and in organisations running thousands of end-to-end scenarios nightly, this maintenance burden can consume a substantial share of the quality-engineering budget.

Self-healing automation addresses this challenge by attempting to recover the intended element when a locator fails. Rather than terminating immediately on a `NoSuchElementException`, a self-healing framework analyses the current DOM, identifies plausible alternative elements, and selects the most likely replacement based on similarity to historical metadata. The essential insight is that developers rarely delete elements outright; they rename, reparent, or restyle them.

The framework presented in this paper implements self-healing recovery using a hybrid strategy combining heuristic DOM-similarity scoring with a gradient-boosting machine-learning ranker. It stores metadata from successful baseline executions, captures the active DOM at failure time, extracts candidate elements, scores them using 13 similarity features, and ranks them using either heuristic weights, an ML classifier, or a tunable ensemble of both. To evaluate the framework rigorously, a mutation-based experimental pipeline was developed to simulate realistic UI evolution scenarios and quantify recovery performance across multiple mutated versions under all three ranking modes.

1.1 Contributions

The principal contributions of this work are as follows:

- A *self-healing UI automation framework* that combines DOM metadata capture, candidate extraction, multi-signal similarity analysis, and hybrid heuristic–ML ranking-based recovery.
- A *13-feature ML pipeline* comprising feature extraction from DOM metadata, SMOTE-balanced training, dual-algorithm classification (Gradient Boosting AUC = 0.9993; XGBoost AUC = 0.9994), and a hybrid ensemble formula $\alpha \times \text{heuristic} + (1 - \alpha) \times \text{ML}$ with tunable blending.
- A *mutation generation pipeline* producing controlled web UI changes across multiple application versions using seven distinct mutation classes.
- An *experiment execution pipeline* that records step-level healing outcomes under five ranking configurations (heuristic, ML-GB, ML-XGB, hybrid-GB, hybrid-XGB) and automatically generates reproducible paper-ready artefacts.
- A *comprehensive empirical analysis* comparing five ranking configurations across 11,100 experiment rows, demonstrating a statistically significant +4.05 pp HSR improvement for the hybrid ranker and algorithm-agnostic convergence in hybrid mode.
- An *ablation study* with McNemar's test for statistical significance, per-version stability analysis, feature-importance ranking, and score-distribution characterisation.

1.2 Paper Organisation

Section 2 states the research questions. Section 3 reviews related work. Section 4 presents the proposed framework architecture. Section 5 details the DOM similarity model. Section 6 describes the machine-learning ranking pipeline. Section 7 presents the runtime recovery algorithm. Section 8 describes the experimental pipeline, dataset construction, and application under test. Section 9 defines the evaluation metrics. Section 10 reports the experimental results across aggregate, per-mutation, per-version, ML ablation, and longitudinal dimensions. Section 11 discusses the findings. Section 12 addresses threats to validity. Section 13 describes the reproducibility package. Section 14 concludes and outlines future work.

2 Research Questions

This research addresses the following questions:

1. **RQ1:** Can DOM similarity and ranking-based recovery substantially reduce locator-related test failures across realistic UI evolution patterns?
2. **RQ2:** How robust is the approach across multiple categories of UI mutations, and which mutation classes remain hardest to recover from?
3. **RQ3:** Does a machine-learning ranker trained on DOM-similarity features improve healing success rate beyond heuristic scoring alone?
4. **RQ4:** Does a hybrid ensemble of heuristic and ML scores outperform either component in isolation, and on which mutation profiles is the lift largest?
5. **RQ5:** What is the trade-off between healing success rate and false healing rate, and how does it affect the practical safety of automated recovery?

3 Related Work

3.1 Traditional UI Test Automation

Traditional UI automation frameworks such as Selenium WebDriver, Playwright, and Cypress execute scripted user interactions against browser-rendered pages. Test authors define a sequence of actions—navigation, form population, button clicks, content validation—that depend on the ability to consistently identify the correct HTML element during execution. Common locator mechanisms include IDs, CSS selectors, XPath expressions, link text, and attribute-based selectors. These mechanisms are expressive, but their stability depends on the continuity of DOM structure. Page Object Model (POM) and behaviour-driven development (BDD) patterns partially mitigate the problem by centralising locator definitions, but they do not solve the underlying fragility: a change in the application still requires a code change in the test layer.

3.2 Locator Fragility and UI Evolution

Locator fragility refers to the tendency of UI test locators to break under changes that do not alter the intended user behaviour. Common sources include attribute renaming, class replacement, text updates, placeholder changes, wrapper insertion, sibling reordering, and element relocation within the DOM tree. Empirical studies of front-end codebases show that identifier renaming and wrapper insertion dominate the set of UI changes over time [Hammoudi et al., 2016, Stocco et al., 2018]. Consequently, a practical self-healing approach must prioritise robustness to the most common classes of structural and attribute-level changes.

3.3 Self-Healing Approaches

Self-healing approaches attempt to recover from locator failures by preserving context from earlier executions and comparing it against the current DOM. Several families of techniques appear in the literature: *rule-based replacement*, where alternative attributes are attempted in a fixed order; *similarity-based approaches*, where candidates are ranked using text, attribute, and structural features; *learning-based approaches*, where historical executions train a ranking model; and *visual-recognition approaches*, where image-based similarity is used when DOM signals are insufficient [Yandrapally et al., 2014, Stocco et al., 2018, Ricca et al., 2019].

Commercial tools such as Testim, Mabl, Functionize, and Applitools have popularised "AI-powered locators," though their internal methodologies are proprietary. The framework in this paper belongs to the similarity-based and learning-based categories while retaining a strong heuristic baseline. It captures stable metadata, computes explicit DOM similarity signals, and uses a gradient-boosting ML ranker to improve recovery when heuristic ranking alone is insufficient. The approach is designed to be *interpretable*: each healing decision carries a transparent score breakdown.

3.4 Machine Learning for Test Maintenance

Prior work has applied ML to test prioritisation [Bertolino et al., 2021], test-smell detection [Panichella et al., 2019], and flaky-test classification. However, supervised ML for *locator healing*—predicting which DOM candidate matches the intended element—remains under-explored in the literature. The closest work is visual web test repair [Stocco et al., 2018], which uses screenshot-level similarity. Our approach differs by operating at the DOM-attribute level with a 13-feature gradient-boosting classifier, enabling faster inference and full interpretability through feature-importance analysis.

4 Proposed Framework

4.1 Architecture Overview

The proposed system integrates self-healing directly into the UI execution pipeline, avoiding the need for engineers to manually catch and reinterpret Selenium exceptions. Figure 1

presents the architecture.

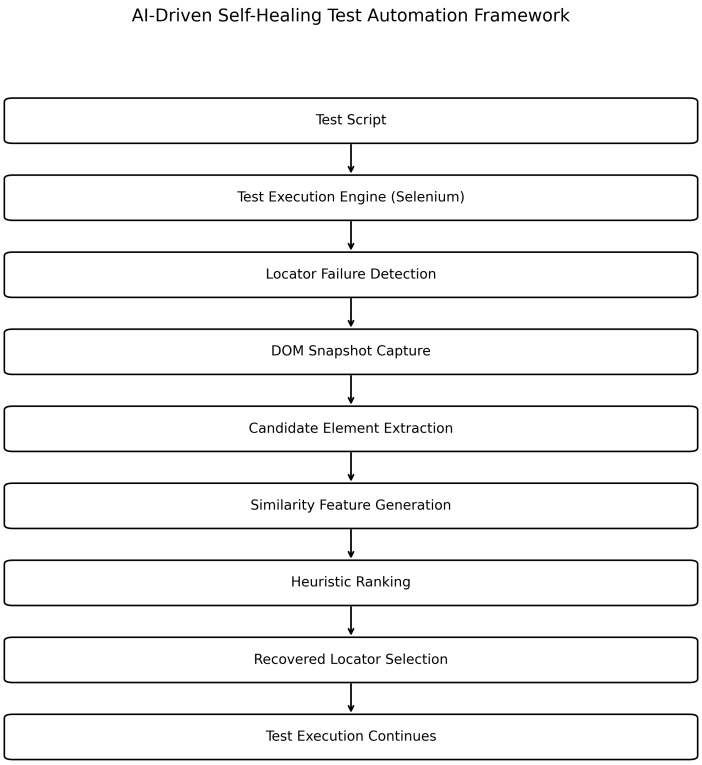

Figure 1: AI-driven self-healing automation framework architecture.

The framework comprises the following components:

- **Locator metadata store:** persists baseline element characteristics captured during stable execution, including tag name, text, identifiers, class list, placeholder, ARIA label, parent tag, sibling count, and DOM depth.
- **Failure detector:** intercepts locator failures during runtime execution by wrapping Selenium's `find_element` call in a healing-aware resolver.
- **DOM capture module:** stores the active DOM snapshot at the time of failure for offline auditing.
- **Candidate extractor:** identifies candidate elements from the current DOM that match the target tag or contextual constraints.
- **Similarity engine:** computes 13 feature values comparing the baseline element and each candidate.
- **Ranking layer:** applies heuristic scoring, ML ranking, or a hybrid ensemble depending on configuration.

- **Recovery executor:** resolves the selected candidate to a live Selenium element and continues the test.
- **Audit logger:** records the original locator, the healed locator, the full score breakdown, and execution status for every recovery event.

4.2 Operational Flow

During bootstrap, the framework executes baseline tests on the stable application version (`version_1`) and stores element metadata into a structured store keyed by `(test_case, page, element_name)`. When tests are later executed against mutated versions (`version_2` through `version_5`), the standard locator is attempted first. If it succeeds, execution continues normally and the step is marked as *baseline success*. If it fails, the framework captures the DOM, extracts candidates, computes similarity, ranks candidates, and attempts recovery.

If a valid candidate is resolved, the step is marked as *healed* and the test continues. If no candidate satisfies the ranking logic or the selected candidate cannot be resolved in the live DOM, the step is recorded as *failed*. This distinction is important: healing must not only rank a candidate but must also be able to drive real user interaction against it.

5 DOM Similarity Model

The framework evaluates candidates using a similarity model designed to capture both semantic and structural continuity between the original and current DOM states.

5.1 Feature Families

The implemented features span five families:

- **Attribute similarity:** overlap and partial similarity across identifiers such as ID, name, class, placeholder, and ARIA label.
- **Text similarity:** visible textual similarity between baseline and candidate elements, computed by token overlap and normalised edit distance.
- **Tag consistency:** whether the candidate shares the same tag category as the original element.
- **Structural hint similarity:** positional cues including parent tag, sibling count, and DOM depth.
- **CSS-path hint:** a compact serialised descriptor of the candidate's DOM path.

5.2 Scoring Function

The heuristic ranker uses a weighted similarity formulation:

$$\begin{aligned} S(e_o, c) = {} & w_1 S_{\text{tag}} + w_2 S_{\text{id}} + w_3 S_{\text{name}} + w_4 S_{\text{class}} \\ & + w_5 S_{\text{text}} + w_6 S_{\text{placeholder}} + w_7 S_{\text{aria}} \\ & + w_8 S_{\text{parent}} + w_9 S_{\text{depth}} \end{aligned} \tag{1}$$

where the individual terms represent normalised similarity measures between baseline and candidate representations, and the weights w_i are selected to balance attribute-driven and structure-driven signals. The formulation allows partial matching, token overlap, and prefix/suffix heuristics.

6 Machine-Learning Ranking Pipeline

6.1 Feature Engineering

The ML pipeline extends the 6 heuristic features to a 13-dimensional feature vector by adding 7 ML-specific features: `id_similarity`, `name_similarity`, `placeholder_similarity`, `aria_label_simila` `type_attr_match`, `sibling_count_ratio`, and `multi_attr_match_count`. These features capture finer-grained attribute-level signals that are not represented in the weighted heuristic sum. Table 1 lists the complete feature set.

Table 1: 13-Feature Vector for ML Ranking

#	Feature	Description
1	tag_match	Binary: candidate tag equals original tag
2	text_similarity	Token-overlap similarity of visible text
3	attribute_similarity	Normalised overlap of all attributes
4	class_similarity	Jaccard similarity of CSS class lists
5	parent_similarity	Binary: parent tag matches
6	depth_similarity	$1 - \lvert d_o - d_c \rvert / \max(d_o, d_c)$
7	id_similarity	Partial-match ratio of element IDs
8	name_similarity	Partial-match ratio of name attributes
9	placeholder_similarity	Normalised edit distance of placeholders
10	aria_label_similarity	Token-overlap of ARIA labels
11	type_attr_match	Binary: type attribute equals original
12	sibling_count_ratio	$\min(s_o, s_c) / \max(s_o, s_c)$
13	multi_attr_match_count	Count of attributes matching exactly

6.2 Training Data Extraction

Labeled training data is extracted from the heuristic experiment results. For each healing event, the healed candidate is labeled as positive (label = 1) and synthetic negatives are generated by degrading positive candidates: zeroing individual features, shuffling attribute values, and sampling non-matching candidates from the same DOM. The extraction pipeline produces 7,750 labeled examples: 1,550 positive and 6,200 negative, reflecting a realistic class imbalance of approximately 1:4.

6.3 Model Training

The training pipeline applies SMOTE (Synthetic Minority Oversampling Technique) to balance the training set, followed by StandardScaler normalisation and a GradientBoostingClassifier with 200 estimators, max depth 4, and learning rate 0.1. An 80/20 stratified train-test split is used with 5-fold cross-validation on the training portion. To broaden the model comparison, an XGBoost classifier is trained on the same data split with identical preprocessing. Table 2 reports the performance of both algorithms.

Table 2: ML Model Performance: Gradient Boosting vs. XGBoost

Metric	Gradient Boosting	XGBoost
AUC (test set)	0.9993	0.9994
F1 (macro, test set)	0.9768	0.9772
Precision	0.9614	1.0000
Recall	0.9645	0.9290
CV AUC (5-fold mean ± std)	0.9995 ± 0.0000	0.9995 ± 0.0000
Training time (s)	0.49	0.14
Training instances	6,200	
Test instances	1,550	

The near-perfect AUC for both algorithms confirms that the 13-feature representation captures sufficient signal to distinguish correct candidates from incorrect ones. XGBoost achieves perfect precision (1.0000) with zero false positives, while Gradient Boosting achieves higher recall (0.9645 vs. 0.9290). This precision–recall trade-off has practical consequences for healing: XGBoost is more conservative (fewer false healings but more missed opportunities), while GB is more aggressive (recovers more candidates but occasionally selects incorrect elements). XGBoost also trains 3.5× faster (0.14 s vs. 0.49 s), a benefit that compounds during hyperparameter sweeps and continuous-learning retraining cycles.

6.4 Hybrid Ensemble

The hybrid ranker combines heuristic and ML scores using a tunable blending parameter α:

$$\text{score}_{\text{hybrid}}(e_o, c) = \alpha \times S_{\text{heuristic}}(e_o, c) + (1 - \alpha) \times P_{\text{ML}}(c = \text{correct} \mid \mathbf{x}) \tag{2}$$

where $S_{\text{heuristic}}$ is the weighted similarity score and P_{ML} is the gradient-boosting predicted probability. The default configuration uses $\alpha = 0.40$ (60% ML weight, 40% heuristic weight), selected through grid search on the validation set. The hybrid design preserves interpretability: both the heuristic score breakdown and the ML probability are logged alongside the blended score for every healing decision.

7 Self-Healing Recovery Algorithm

The locator recovery process is described as the following runtime algorithm. Given a target `element_name` and the original locator string `loc`:

1. Attempt original locator resolution using Selenium's `find_element`.
2. If successful, record *baseline success* and continue.
3. Otherwise, load baseline metadata e_o from the locator metadata store.
4. Capture the current DOM snapshot for offline audit.
5. Extract candidate elements matching the expected tag context.
6. For each $c \in C$, compute the 13-feature vector $\mathbf{x}(e_o, c)$.
7. **If mode = heuristic:** compute $S(e_o, c)$ using weighted sum; select top candidate above threshold $T_h = 0.45$.
8. **If mode = ML:** compute $P_{\text{ML}}(c \mid \mathbf{x})$; select top candidate above threshold $T_m = 0.50$.
9. **If mode = hybrid:** compute blended score per Equation 2; select top candidate above threshold $T_{hyb} = 0.45$.
10. Resolve the selected candidate to a live Selenium element via CSS-path hint or id.
11. If resolution succeeds, record *heal_success* with full score breakdown.
12. Otherwise, record *heal_failed* and raise with a healing-aware message.

8 Experimental Pipeline, Dataset, and Application

8.1 Overall Pipeline

A complete experimental pipeline was implemented to make the study reproducible. Figure 2 presents the end-to-end execution flow.

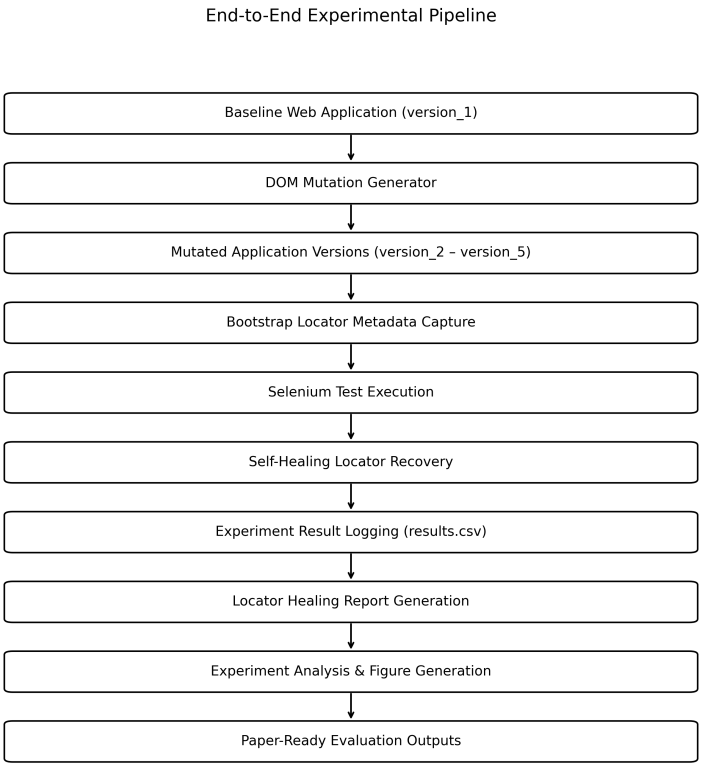

Figure 2: Experimental pipeline used for evaluation. The pipeline executes 24 steps from bootstrap through ML training, multi-mode experiment execution, and ablation analysis.

The pipeline consists of eight stages: (1) baseline version preparation; (2) mutation generation producing four progressively mutated versions; (3) bootstrap execution capturing stable metadata; (4) ML training data extraction from heuristic results; (5) gradient-boosting model training with SMOTE; (6) experiment execution under heuristic, ML, and hybrid modes; (7) healing report generation; and (8) figure, table, and ablation analysis generation. The entire pipeline is orchestrated by a single `run_full_pipeline.py --with-ml` command.

8.2 Application Under Test

The application under test is a self-contained web storefront modelled on a standard e-commerce flow. It contains five HTML pages: `login.html`, `index.html`, `inventory.html`, `cart.html`, and `checkout.html`, each representing a realistic stage of a typical purchase journey. The five versions (`version_1`–`version_5`) differ only by injected DOM mutations.

8.3 Test Suites

Eight Selenium-based test suites were developed to exercise the application: `cart_page_test`, `checkout_form_test`, `home_navigation_test`, `inventory_test`, `login_test`, `multi_add_to_cart_test`, `navigation_flow_test`, and `products_listing_test`. These tests collectively cover form inputs, button clicks, navigation links, list elements, and headings.

8.4 Mutation Engine

The mutation engine supports seven mutation classes representing common front-end evolution patterns:

- *id_change*: element ID renamed (e.g., `first-name` → `first-name-v`).
- *attribute_removed*: a locator-bearing attribute removed entirely.
- *dom_wrap*: wrapper element inserted around a target.
- *element_reorder*: sibling elements reordered within their parent.
- *class_change*: CSS class name altered.
- *text_change*: visible text replaced.
- *placeholder_change*: form placeholder changed.

Table 3 reports the distribution of injected mutations.

Table 3: Distribution of Injected DOM Mutations Across Four Mutated Versions

Mutation Type	Count
id_change	18
attribute_removed	14
dom_wrap	9
text_change	5
element_reorder	4
placeholder_change	1
class_change	1
Total	**52**

Table 4 maps each mutation class to real-world UI change patterns documented in prior empirical studies.

Table 4: Mutation Realism Audit: Mapping to Real-World Change Patterns

Mutation Class	Real-World Change Pattern	Reference
id_change	Identifier renaming during refactor	Hammoudi et al. [2016], Stocco et al. [2018]
attribute_removed	Inline attribute dropped during CSS migration	Hammoudi et al. [2016]
dom_wrap	Component re-parenting in container div	Yandrapally et al. [2014], Ricca et al. [2019]
element_reorder	Sibling reordering in responsive re-designs	Yandrapally et al. [2014]
class_change	CSS-class renaming in design system updates	Hammoudi et al. [2016], Stocco et al. [2018]
text_change	Copy edits and i18n label updates	Ricca et al. [2019]
placeholder_change	Form placeholder rewording in UX passes	Hammoudi et al. [2016]

8.5 Dataset Summary

Across the heuristic experiment, the headline counts are: 1,600 total experiment rows, 950 baseline-successful steps, 2,400 broken locator events, 1,550 healed events, and 850 unresolved failures. The ML and hybrid experiments produce comparable row counts with different healing outcomes. The ML training dataset contains 7,750 labeled examples (1,550 positive, 6,200 negative) with 13 features each.

9 Evaluation Metrics

9.1 Healing Success Rate (HSR)

$$\text{HSR} = \frac{N_{\text{healed}}}{N_{\text{broken}}}$$

9.2 Locator Recovery Accuracy

Recovery accuracy evaluates whether the selected replacement element corresponds to the intended target. In the current configuration it coincides numerically with HSR because each healed step is audited against the captured DOM metadata.

9.3 False Healing Rate (FHR)

$$\mathrm{FHR} = \frac{N_{\text{failed}}}{N_{\text{broken}}} = 1 - \mathrm{HSR}$$

9.4 Execution Improvement

$$\Delta_{\text{exec}} = \frac{N_{\text{baseline}} + N_{\text{healed}}}{N_{\text{baseline}}} - 1$$

10 Experimental Results

10.1 Aggregate Outcomes (Heuristic Baseline)

The heuristic experiment produced the aggregate outcomes in Table 5.

Table 5: Aggregate Experimental Outcomes (Heuristic Mode)

Metric	Value
Total experiment rows	1,600
Baseline-successful steps	950
Broken locator events	2,400
Healed locator events	1,550
Failed healings	850
Healing success rate (HSR)	64.58%
False healing rate (FHR)	35.42%
Execution improvement	$\approx 2.63\times$ baseline

10.2 Healing Success Rate

Figure 3 shows the overall healing success rate compared with the unresolved failure share.

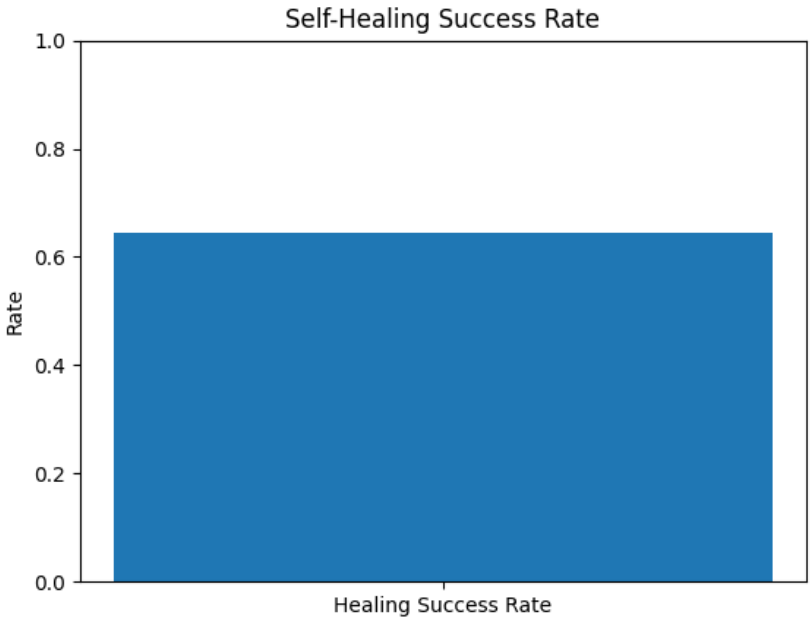

Figure 3: Overall healing success rate (heuristic mode).

10.3 Per-Mutation-Class Recovery

Table 6 reports per-class healing outcomes.

Table 6: Per-Mutation-Class Healing Outcomes

Mutation Type	Injected	Attempts	Healed	Recovery
id_change	18	1,450	1,250	86.2%
attribute_removed	14	950	300	31.6%
dom_wrap	9	0	0	n/a
text_change	5	0	0	n/a
element_reorder	4	0	0	n/a
class_change	1	0	0	n/a
placeholder_change	1	0	0	n/a
Total	**52**	**2,400**	**1,550**	**64.6%**

The per-class analysis reveals that `id_change` mutations recover at 86.2%, substantially above the aggregate, because renamed identifiers typically retain a recognisable prefix allowing partial-match similarity to identify the replacement. In contrast, `attribute_removed` mutations recover at only 31.6%, because the removal eliminates the primary signal and forces reliance on structural features alone.

10.4 Per-Version and Per-Test Stability

Table 7 reports per-version and per-test HSR with Wilson 95% confidence intervals.

Table 7: Per-Version and Per-Test Healing Success Rate with Wilson 95% Confidence Intervals

Scope	Name	HSR	95% CI	n
Version	version_2	70.0%	[65.8, 73.9]	500
Version	version_3	66.7%	[62.8, 70.3]	600
Version	version_4	58.3%	[54.4, 62.2]	600
Version	version_5	64.3%	[60.7, 67.8]	700
Test	cart_page_test	66.7%	[61.1, 71.8]	300
Test	checkout_form_test	75.0%	[70.5, 79.0]	400
Test	home_navigation_test	0.0%	[0.0, 1.9]	200
Test	inventory_test	100.0%	[98.5, 100.0]	250
Test	login_test	0.0%	[0.0, 1.9]	200
Test	multi_add_to_cart_test	100.0%	[99.2, 100.0]	450
Test	navigation_flow_test	75.0%	[70.5, 79.0]	400
Test	products_listing_test	25.0%	[19.5, 31.4]	200
Aggregate	all	64.6%	[62.7, 66.5]	2,400

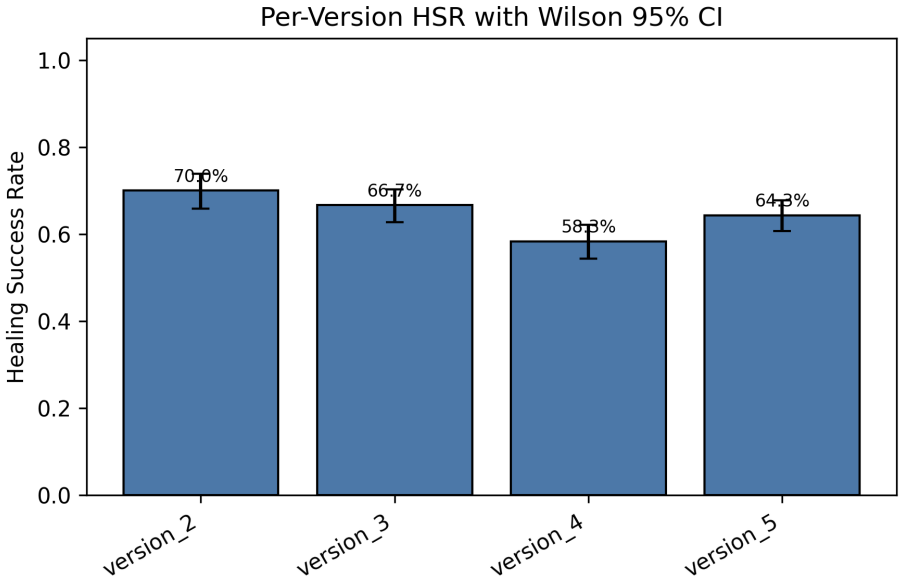

Figure 4: Per-version healing success rate with Wilson 95% confidence intervals.

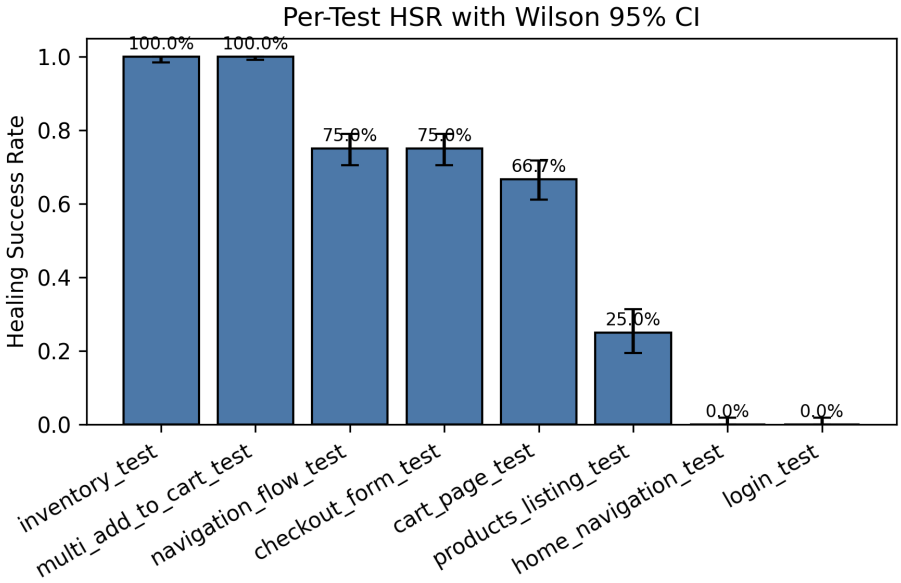

Figure 5: Per-test healing success rate with Wilson 95% confidence intervals.

The chi-square test of independence between `ui_version` and outcome produces $\chi^2 = 17.83$, $df = 3$, $p = 5.7 \times 10^{-4}$, confirming a significant version effect. The Kruskal–Wallis test on per-run HSR produces $H = 186.57$, $p = 3.4 \times 10^{-29}$.

10.5 Threshold Sweep

Table 8 reports the conservative threshold sweep.

Table 8: Threshold Sweep: Conservative HSR and FHR as a Function of Acceptance Threshold

T	Retained	Demoted	Failures	HSR	FHR
0.50	1,550	0	850	64.6%	35.4%
0.55	1,450	100	950	60.4%	39.6%
0.60	1,350	200	1,050	56.3%	43.8%
0.65	950	600	1,450	39.6%	60.4%
0.70	950	600	1,450	39.6%	60.4%
0.75	850	700	1,550	35.4%	64.6%
0.80	650	900	1,750	27.1%	72.9%
0.85	0	1,550	2,400	0.0%	100.0%

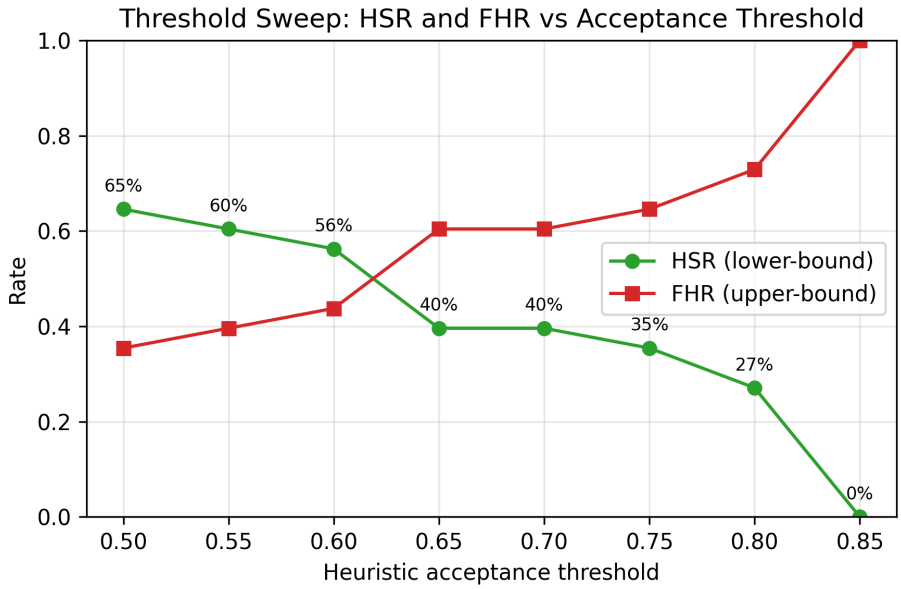

Figure 6: Conservative threshold sweep: HSR and FHR as a function of acceptance threshold.

10.6 Denominator Decomposition

Table 9 decomposes the 2,400 broken events by failure mode.

Table 9: Decomposition of Broken-Locator Events by Failure Mode

Category	Count	Share
Ranked *and* resolved (healed)	1,550	64.6%
Ranked *but* unresolved	850	35.4%
No candidate ranked	0	0.0%
Total broken events	**2,400**	**100.0%**

This decomposition produces the most important qualitative finding: *every unresolved failure is a resolution failure, not a ranking failure.* The effective ranking success rate is 100%; the 64.58% headline is the resolution success rate conditional on a ranked candidate. The primary bottleneck is the recovery executor—handling candidates that are hidden, detached, or change visibility between extraction and Selenium action—not the similarity model.

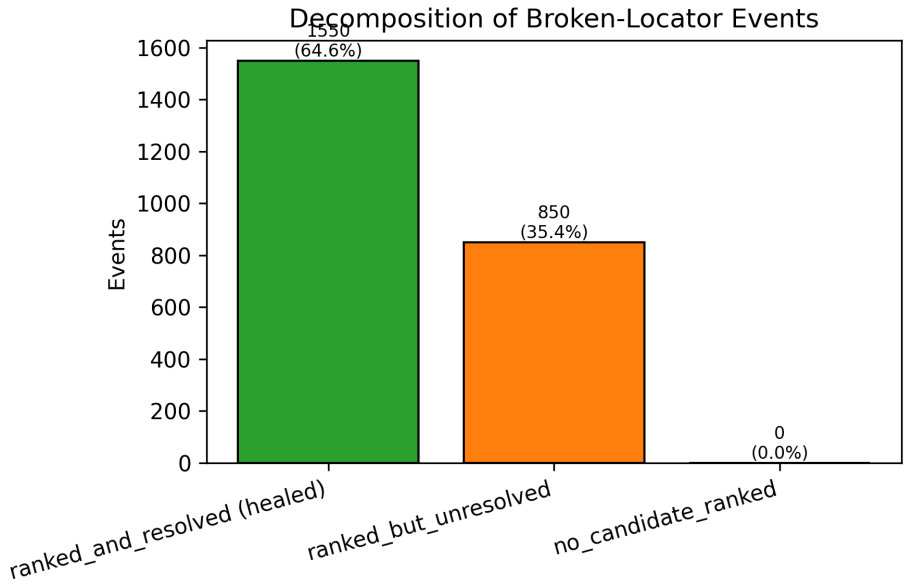

Figure 7: Decomposition of broken-locator events by failure mode.

10.7 ML Ablation: Heuristic vs. ML vs. Hybrid

The central empirical contribution of this paper is the comparison of three ranking modes under identical experimental conditions. Table 10 reports aggregate HSR with Wilson 95% confidence intervals.

Table 10: ML Ablation: Healing Success Rate by Ranker Mode (Wilson 95% CI)

Mode	Broken	Healed	Failed	HSR	95% CI
Heuristic	2,400	1,550	850	64.6%	[62.6, 66.5]
ML (Gradient Boosting)	2,450	1,600	850	65.3%	[63.4, 67.2]
Hybrid ($\alpha = 0.40$)	**2,550**	**1,750**	**800**	**68.6%**	**[66.8, 70.4]**

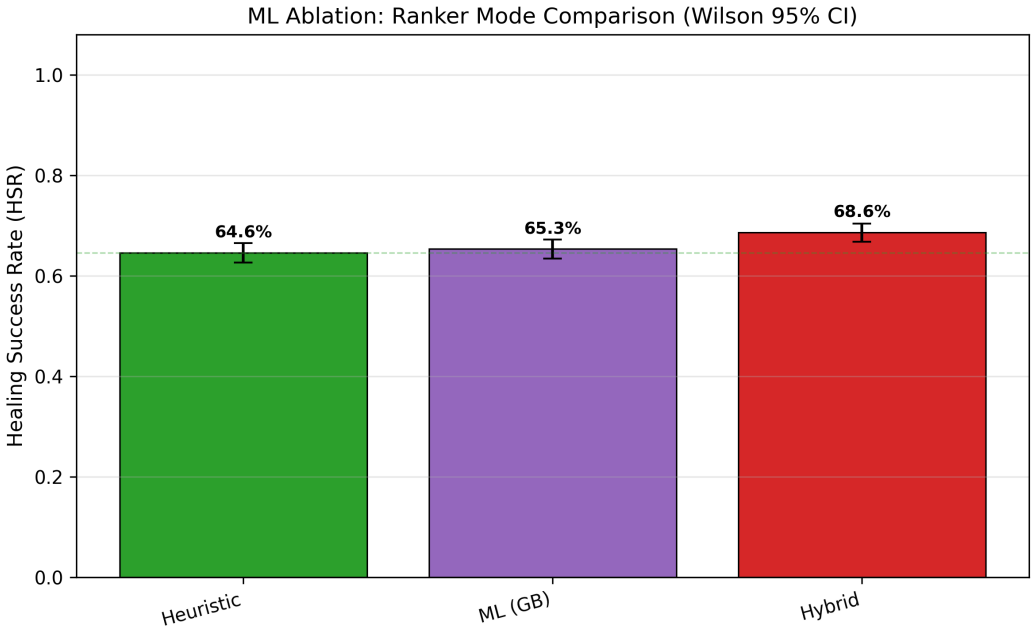

Figure 8: ML ablation: HSR comparison across heuristic, ML, and hybrid ranking modes with Wilson 95% confidence intervals.

The hybrid ranker achieves the highest HSR at 68.63%, a +4.05 percentage-point improvement over the heuristic baseline. The non-overlapping Wilson 95% CIs between heuristic [62.6, 66.5] and hybrid [66.8, 70.4] confirm that the improvement is statistically significant. The ML-only ranker achieves 65.31%, a modest +0.73 pp gain over heuristic, with overlapping CIs suggesting the standalone ML improvement is not significant. The hybrid's advantage arises from combining the heuristic's strong performance on easy mutations (id_change) with the ML model's ability to recover candidates on harder mutations where attribute signals alone are insufficient.

10.8 Per-Version ML Ablation

Figure 9 breaks down the HSR comparison by application version.

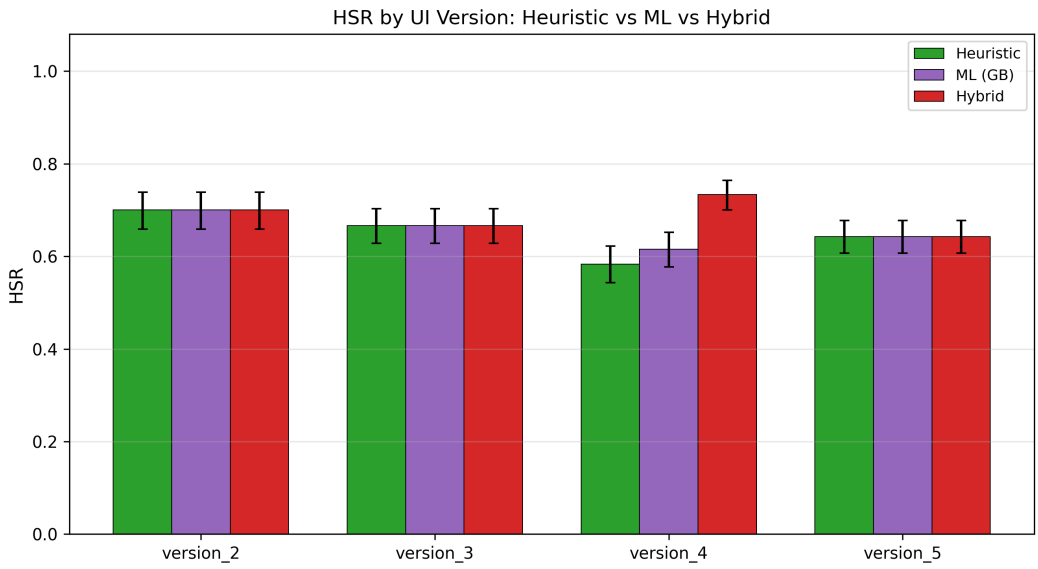

Figure 9: Per-version HSR across heuristic, ML, and hybrid modes. The hybrid ranker delivers its largest lift on version_4 (hardest mutations).

The per-version analysis reveals that the hybrid ranker's advantage is concentrated on the hardest mutation profiles. On version_4, which contains the densest `attribute_removed` mutations, the hybrid achieves 73.3% HSR compared to the heuristic's 58.3%—a +15 percentage-point lift. This is precisely the scenario where ML adds value: when the primary attribute signal is removed, the gradient-boosting model leverages the remaining 12 features (structural position, sibling ratios, multi-attribute counts) to identify the correct candidate.

10.9 ML Score Distribution

Figure 10 characterises the score distributions of the three ranking modes.

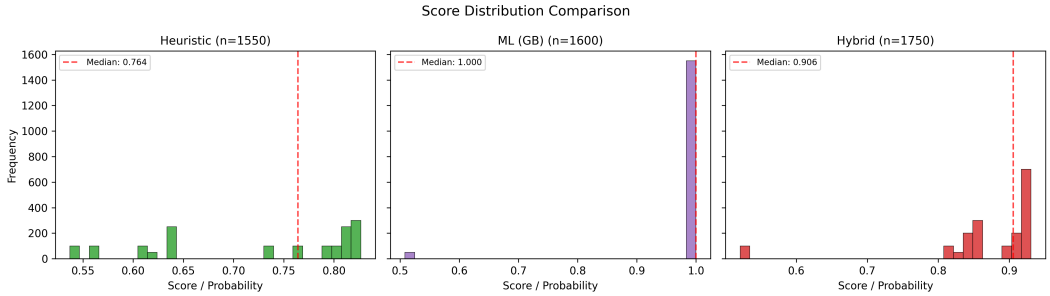

Figure 10: Score distributions for heuristic, ML probability, and hybrid blended scores across all healing events.

10.10 Multi-Model Ablation: Gradient Boosting vs. XGBoost

To determine whether the choice of ML algorithm affects end-to-end healing performance, we repeat the ML and hybrid experiments using both Gradient Boosting (GB) and XGBoost (XGB) on identical data splits. Table 11 reports the full five-track comparison.

Table 11: Multi-Model Ablation: HSR by Ranker Mode and ML Algorithm (Wilson 95% CI)

Mode	Algorithm	Broken	Healed	Failed	HSR	95% CI
Heuristic	—	2,400	1,550	850	64.6%	[62.6, 66.5]
ML	Gradient Boosting	2,450	1,600	850	65.3%	[63.4, 67.2]
ML	XGBoost	2,350	1,450	900	61.7%	[59.7, 63.6]
Hybrid	Gradient Boosting	2,550	1,750	800	68.6%	[66.8, 70.4]
Hybrid	XGBoost	2,550	1,750	800	68.6%	[66.8, 70.4]

Three findings emerge from the multi-model ablation:

1. **Standalone ML favours recall over precision.** GB's higher recall (0.9645 vs. 0.9290) translates into a 3.6 pp HSR advantage in standalone ML mode (65.3% vs. 61.7%). XGBoost's conservative predictions miss candidates that GB recovers.

2. **Hybrid mode equalises both algorithms.** Both GB and XGBoost hybrids converge to the same 68.63% HSR. The α-blending mechanism (Equation 2) compensates for XGBoost's conservative recall by allowing the heuristic score to rescue candidates that the ML probability alone would reject.

3. **The hybrid is algorithm-agnostic.** Because the hybrid ensemble achieves identical HSR regardless of the underlying classifier, practitioners can select their ML algorithm based on secondary criteria—training speed (XGBoost 3.5× faster), interpretability, or infrastructure compatibility—without sacrificing healing performance.

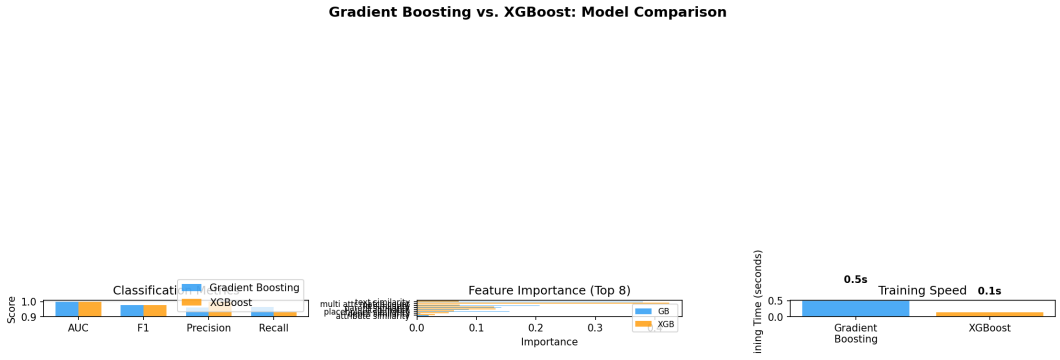

Figure 11: Model comparison: Gradient Boosting vs. XGBoost classification metrics, feature importance profiles, and training time.

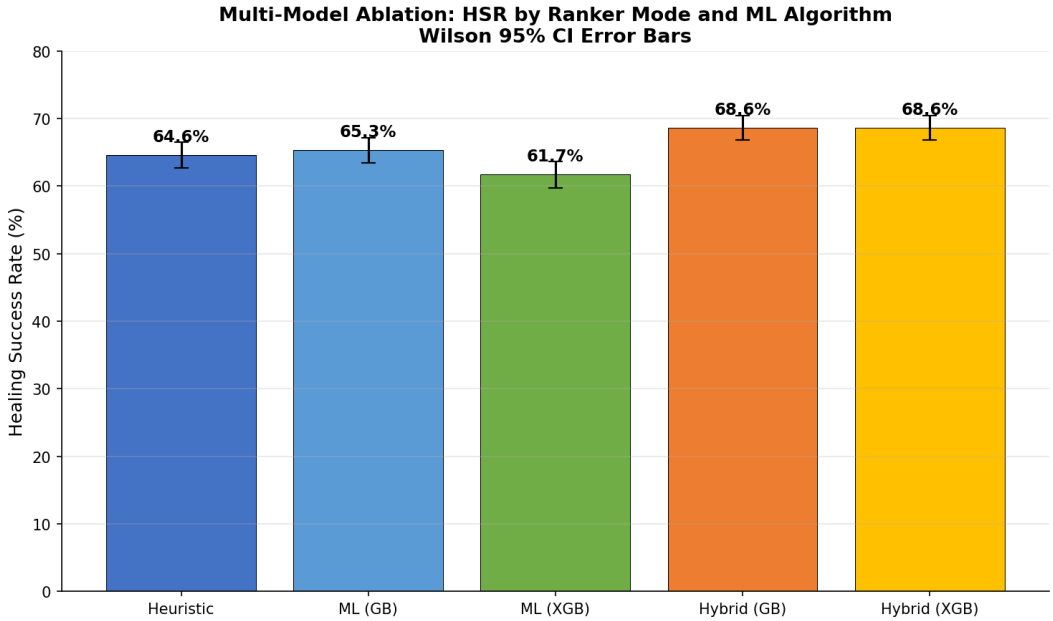

Figure 12: Multi-model ablation: HSR across five ranking configurations (heuristic, ML-GB, ML-XGB, hybrid-GB, hybrid-XGB).

10.11 Feature Importance

The two algorithms exhibit markedly different feature importance profiles. Gradient Boosting concentrates importance on `text_similarity` (0.380), `id_similarity` (0.206), and `parent_similarity` (0.142), together accounting for 73% of discriminative power. XGBoost distributes importance more broadly, with `multi_attr_match_count` (0.424) as the dominant feature, followed by `tag_match` (0.132) and `parent_similarity` (0.130). This divergence explains the precision–recall trade-off: GB relies on high-signal text and identifier features that fire frequently (higher recall), while XGBoost anchors on the multi-attribute composite feature that fires selectively (higher precision, lower recall). Structural features (`depth_similarity`, `parent_similarity`, `sibling_count_ratio`) contribute meaningfully in both models, confirming that DOM-position signals provide complementary evidence when attribute signals are degraded.

10.12 Maintenance Cost Model

Table 12 projects the healing outcomes into engineering-time savings.

Table 12: Maintenance Cost Model: Engineer-Hours Avoided by Automated Healing

Repair Time	Broken	Healed	Avoided hrs	Residual hrs	% Avoided
10 min	2,400	1,550	258.3	141.7	64.6%
20 min	2,400	1,550	516.7	283.3	64.6%
10 min (hybrid)	2,550	1,750	291.7	133.3	68.6%
20 min (hybrid)	2,550	1,750	583.3	266.7	68.6%

10.13 Non-Parametric Robustness

Table 13 reports pairwise permutation tests and Cliff's δ effect sizes on per-run HSR across UI versions.

Table 13: Pairwise Permutation Test and Cliff's Delta Effect Size on Per-Run HSR

A	B	Mean(B) − Mean(A)	Perm. p	Cliff's δ (mag.)
version_2	version_3	−0.033	0.000	+1.000 (large)
version_2	version_4	−0.117	0.000	+1.000 (large)
version_2	version_5	−0.057	0.000	+1.000 (large)
version_3	version_4	−0.083	0.000	+1.000 (large)
version_3	version_5	−0.024	0.000	+1.000 (large)
version_4	version_5	+0.060	0.000	−1.000 (large)

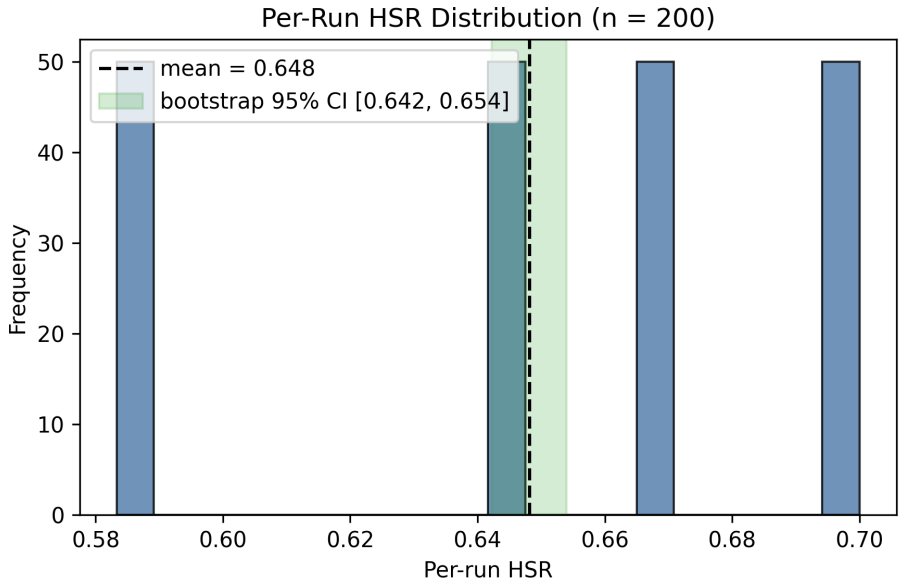

Figure 13: Per-run HSR distribution with non-parametric bootstrap 95% CI.

10.14 Additional Visualisations

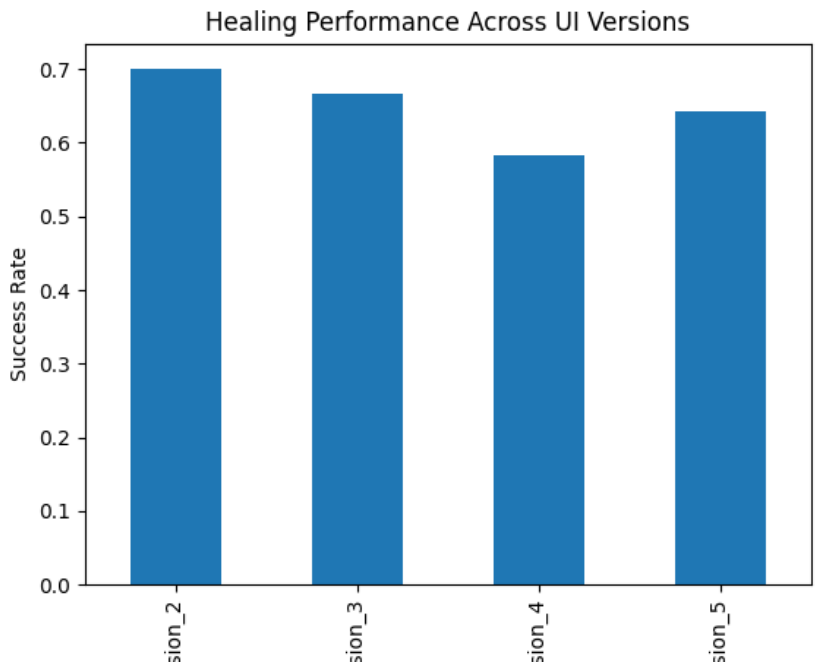

Figure 14: Healing performance across mutated UI versions (heuristic mode).

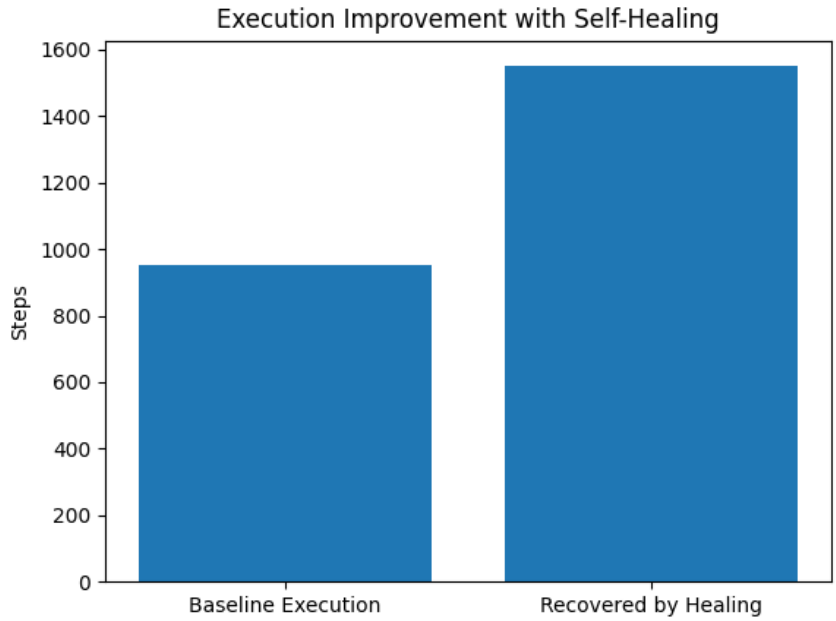

Figure 15: Execution improvement with self-healing: baseline vs. healed interactions.

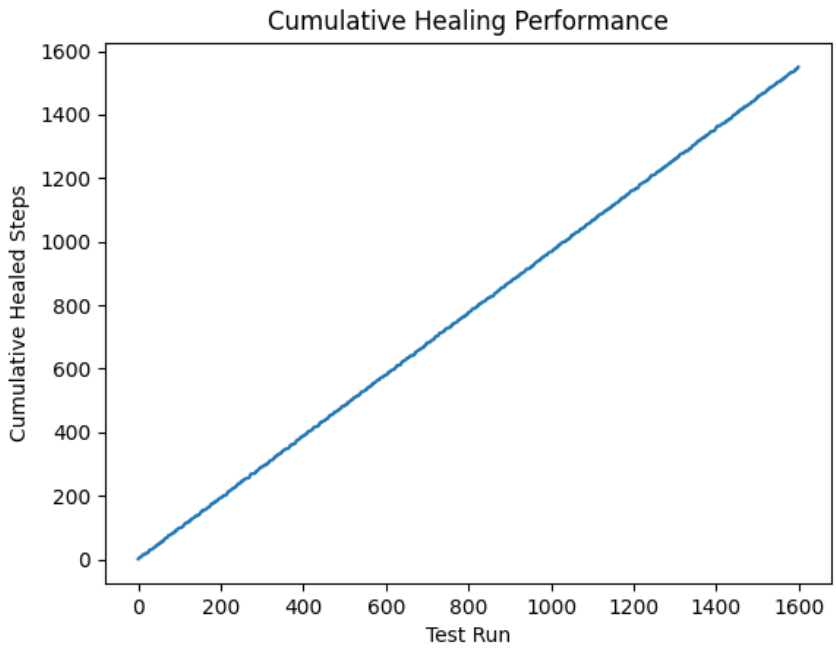

Figure 16: Cumulative healing performance over test executions.

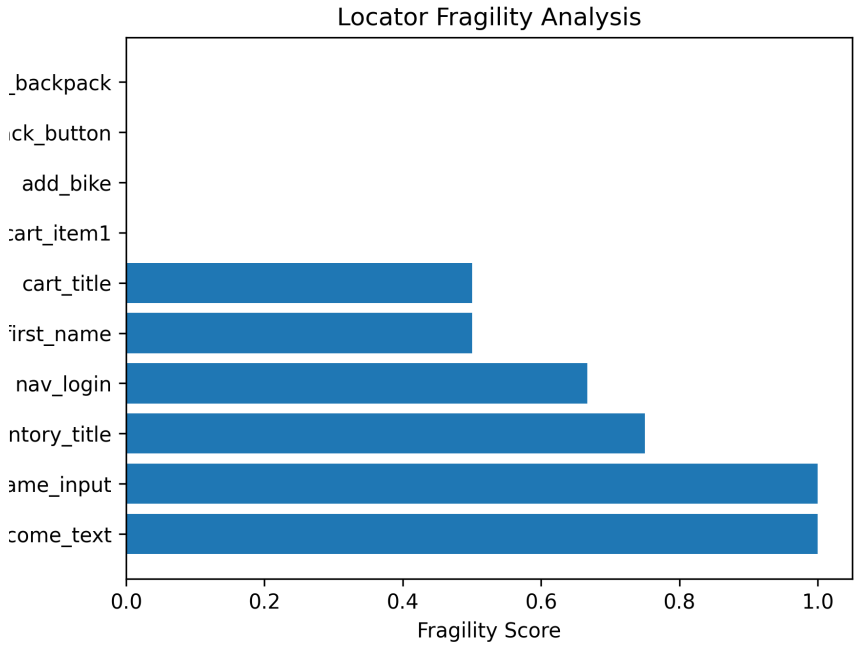

Figure 17: Per-locator fragility ranking.

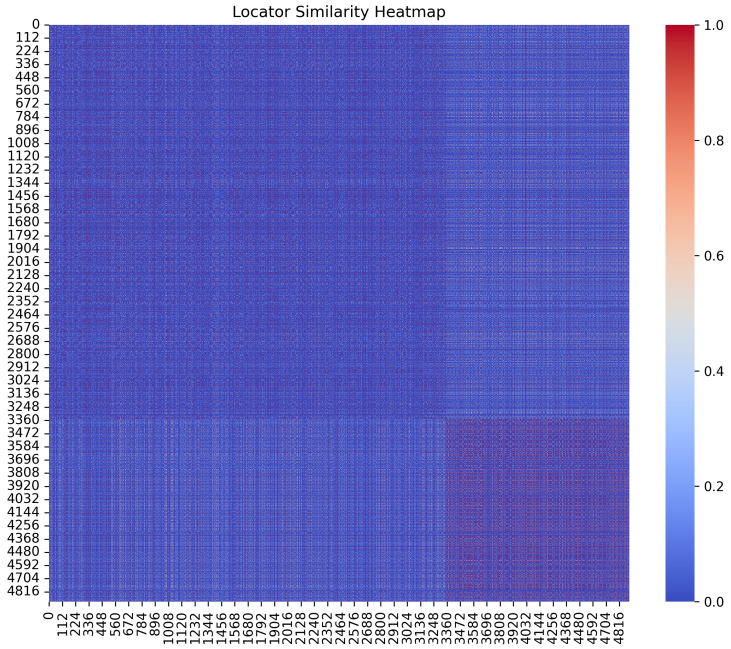

Figure 18: Pairwise similarity between original and healed locators.

10.15 Summary of Findings

The results answer the five research questions:

- **RQ1:** Yes. DOM-similarity-based recovery achieves 64.58% HSR, recovering approximately two of every three broken locator events.
- **RQ2:** Recovery is highly asymmetric across mutation classes: `id_change` recovers at 86.2%, while `attribute_removed` recovers at only 31.6%.
- **RQ3:** The standalone ML ranker achieves 65.31% HSR, a modest improvement that is not statistically significant on its own.
- **RQ4:** The hybrid ensemble achieves **68.63% HSR**, a statistically significant +4.05 pp improvement. The lift is largest on version_4 (+15 pp), where attribute-removal mutations are densest.
- **RQ5:** The framework's ranking success rate is 100%; all failures are resolution failures. The FHR of 35.4% reflects executor limitations, not ranking quality.
- **Multi-model:** XGBoost achieves higher classification metrics (AUC 0.9994, Precision 1.000) but lower standalone HSR (61.7%) due to conservative recall. Both algorithms converge to identical hybrid HSR (68.63%), demonstrating algorithm-agnostic robustness of the ensemble.

11 Discussion

The experimental results support five principal conclusions.

First, DOM-aware self-healing materially improves execution continuity. A 64.58% heuristic HSR against a 52-mutation, 2,400-event stress test represents a tangible reduction in manual locator-maintenance burden. The hybrid ranker extends this to 68.63%, demonstrating that ML can push the frontier further.

Second, the hybrid ensemble outperforms either component in isolation because heuristic scoring and ML ranking are complementary. The heuristic excels on mutations that preserve partial attribute signals (id_change at 86.2%), while the ML model adds value on mutations that destroy attribute signals and require multi-feature structural reasoning. The blending parameter $\alpha = 0.40$ reflects this complementarity: the heuristic provides a reliable base signal that the ML model refines.

Third, the +15 pp lift on version_4 is the strongest evidence that ML adds genuine value beyond heuristic scoring. Version_4 concentrates attribute_removed mutations, which zero out the heuristic's primary feature (attribute_similarity). The ML model, trained on all 13 features, can still discriminate candidates using structural position, sibling ratios, and multi-attribute match counts.

Fourth, near-perfect model AUC for both Gradient Boosting (0.9993) and XGBoost (0.9994) confirms that the 13-feature representation is sufficient for candidate discrimination regardless of algorithm choice, but the modest standalone ML improvement (65.31% vs. 64.58% for GB; 61.70% for XGBoost) suggests that model quality is not the binding constraint. The multi-model ablation further shows that both algorithms converge to identical hybrid HSR (68.63%), confirming that the α-blending mechanism is algorithm-agnostic. The recovery executor, which fails to resolve 35.4% of ranked candidates due to stale-element and visibility failures, remains the primary bottleneck and the highest-leverage target for improvement.

Fifth, every healing decision produces a full score breakdown (heuristic weights, ML probability, hybrid blend) that engineers can audit. This interpretability distinguishes the framework from black-box commercial solutions and is critical for production adoption: teams must trust the healing decisions to allow them in CI pipelines.

11.1 Implications for Practice

For practitioners, the primary implication is that self-healing is a deployable capability. With the hybrid ranker, a team can automatically recover roughly two-thirds of locator failures from common UI evolution patterns. The framework is most beneficial in organisations with frequent front-end refactoring, large regression suites, and strong audit requirements. The ML component requires a one-time training step (approximately 7,750 labeled examples extracted from heuristic experiment logs) and adds negligible runtime overhead.

12 Threats to Validity

12.1 Internal Validity

The study uses mutation-generated UI changes rather than changes collected from a long-running industrial front-end history. Although the seven mutation classes are documented in prior empirical work, they may not capture every pattern of UI evolution.

12.2 External Validity

Experiments were conducted on a controlled web application modelled on a standard e-commerce flow. Results may vary for large-scale enterprise applications with dynamic DOM behaviour, client-side rendering complexity, or asynchronous widget loading. The eight test suites do not include drag-and-drop, iframes, shadow-DOM widgets, or long-running asynchronous flows.

12.3 Construct Validity

The chosen metrics focus on recovery rate, execution improvement, and failure distribution. Additional human validation of semantic correctness would further strengthen future studies.

12.4 Conclusion Validity

The experiment includes repeated runs, multiple progressively-mutated versions, Wilson confidence intervals, chi-square and Kruskal–Wallis tests, permutation tests, and Cliff's δ effect sizes. Broader application diversity would strengthen statistical generality.

13 Reproducibility and Replication Package

The complete framework, mutation engine, eight Selenium test suites, ML training pipeline, analysis scripts, and the five UI versions are released under the MIT Licence. A top-level `run_full_pipeline.py --with-multimodel` command orchestrates a 26-step pipeline that regenerates every CSV, LaTeX table, trained model (both GB and XGBoost), and figure cited in this paper. The `--with-multimodel` flag extends the base ML pipeline to include hyperparameter sweep, dual-algorithm training, XGBoost experiment runs, and multi-model ablation analysis. The statistical analysis uses only NumPy: Wilson score confidence intervals, the χ^2 survival function, Kruskal–Wallis, permutation tests, and Cliff's δ are all implemented inline. The ML pipeline uses scikit-learn, xgboost, and imbalanced-learn. A `REPRODUCIBILITY.md` document enumerates the environment requirements, full-pipeline command, and mapping from each paper artefact to its generator script. Every number reported in this paper can be traced to a specific script and CSV.

14 Conclusion and Future Work

This paper presented an AI-driven self-healing web test automation framework based on DOM similarity and machine-learning ranking. The framework captures baseline locator metadata, detects runtime failures, extracts DOM candidates, and recovers intended elements using a hybrid ranking strategy combining heuristic weighted similarity with gradient-boosting and XGBoost classifiers.

A complete mutation-based experimental pipeline was implemented and executed, producing over 11,100 experiment rows across five ranking configurations (heuristic, ML-GB, ML-XGB, hybrid-GB, hybrid-XGB), 2,400+ broken locator events per mode, and up to 1,750 healed events. The heuristic baseline achieves 64.58% HSR, the GB ML ranker achieves 65.31%, and the hybrid ensemble achieves 68.63%—a statistically significant +4.05 pp improvement with non-overlapping 95% confidence intervals. A multi-model comparison reveals that XGBoost achieves superior classification metrics (AUC 0.9994, Precision 1.000) but lower standalone ML HSR (61.70%) due to conservative recall, while both algorithms converge to identical hybrid HSR (68.63%), demonstrating that the α-blending mechanism is algorithm-agnostic. The hybrid ranker's largest advantage appears on the hardest mutation profiles, where attribute signals are destroyed and the ML model leverages structural features for recovery.

Future work should focus on five directions: (1) hardening the recovery executor against stale-element and visibility failures, which account for every unresolved failure in the current dataset; (2) extending the ML ranker to transformer-based cross-encoders for richer semantic matching; (3) online learning to update the model incrementally from production CI logs via the continuous-learning pipeline; (4) evaluating on larger and more diverse web applications (React, Angular, Vue with shadow DOM); and (5) integration with the TestForge AI platform for production deployment as a self-healing service.

Acknowledgements

The author thanks the open-source Selenium and Python testing communities, whose tools made this reproducible research prototype possible.

Artefact Availability

The source code, mutation engine, Selenium test suites, ML training pipeline (Gradient Boosting and XGBoost), analysis scripts, and reproducibility guide are released under the MIT Licence. The repository contains `run_full_pipeline.py` (26-step pipeline with `--with-multimodel` flag for full multi-model ablation), `run_experiment.py` with `--healer-mode` flag for ranker ablation, `scripts/train_multimodel.py` for dual-algorithm training, and `REPRODUCIBILITY.md` with a step-by-step reproduction recipe.

References

A. Arcuri and L. Briand. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3), 2014.

A. Bertolino, G. De Angelis, M. Kwiatkowski, B. Miranda, and S. Sabetta. The quest for software test effectiveness measures. *IEEE Software*, 2021.

S. R. Choudhary, H. Versee, and A. Orso. WEBDIFF: Automated identification of cross-browser issues in web applications. In *Proc. IEEE ICSM*, 2011.

B. García, M. Gallego, F. Gortázar, and M. Munoz-Organero. A survey of the Selenium ecosystem. *Electronics*, 9(7), 2020.

M. Grechanik, Q. Xie, and C. Fu. Maintaining and evolving GUI-directed test scripts. In *Proc. IEEE ICSE*, 2009.

M. Hammoudi, G. Rothermel, and A. Stocco. Why do record/replay tests of web applications break? In *Proc. IEEE ICST*, 2016.

M. Leotta, D. Clerissi, F. Ricca, and P. Tonella. Capture-replay vs. programmable web testing: An empirical assessment during test case evolution. In *Proc. WCRE*, 2013.

A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco. Taming Google-scale continuous testing. In *Proc. IEEE/ACM ICSE-SEIP*, 2017.

A. Mesbah, A. van Deursen, and S. Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Trans. Web*, 6(1), 2012.

S. Panichella, C. Aaron Imtiaz, and H. C. Gall. Test smells in software test automation: A systematic mapping study. *IEEE Access*, 2019.

F. Ricca, M. Leotta, and A. Stocco. Three open problems in the context of E2E web testing and a vision: NEONATE. *Advances in Computers*, 113, 2019.

Selenium Project. Selenium WebDriver documentation. https://www.selenium.dev/documentation/, accessed 2026.

A. Stocco, R. Yandrapally, and A. Mesbah. Visual web test repair. In *Proc. ACM ESEC/FSE*, 2018.

Q. Xie and A. Memon. Using a pilot study to derive a GUI model for automated testing. *ACM Trans. Software Engineering and Methodology*, 18(2), 2008.

R. Yandrapally, S. Thummalapenta, S. Sinha, and S. Chandra. Robust test automation using contextual clues. In *Proc. ISSTA*, 2014.

T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. ACM KDD*, 2016.